PHASE 1 · FOUNDATIONS

Day 02 — Blocks, chains & Proof of Work

Learning Blockchain in 15 Days · Day 2 of 15 · Friday 16 October 2026 · Manish Khadka

Yesterday's hash becomes today's glue: put data in a block, link every block to the hash of the one before it, and make each block expensive to create. That is how a ledger becomes tamper-evident and tamper-resistant.

Learning objectives

1. Explain what a block header contains and why each block stores the previous block's hash.
2. Explain Proof of Work: what the nonce and the difficulty are, and why the expected work is about 2^d hashes.
3. Build a chain validator that reports *where* and *why* a chain breaks, and list the rules it must enforce.
4. Show, with real numbers, why rewriting history means re-mining every later block.
5. Recognise the classic bugs: unchecked targets, self-declared difficulty, ambiguous encodings, non-canonical JSON.

The concept, step by step

Step 1: a block is a box with a label

The box holds data (here: a payment). The label is the SHA-256 hash of the block's **header**. Change anything inside the box and the label changes completely (Day 01's avalanche effect).

Step 2: every box holds the previous box's label

Each header contains `prevHash`, the hash of the block before it. That makes a chain: block 2 depends on block 1, which depends on block 0, the fixed **genesis block**.



Figure 1 — Real blocks from today's run (16-bit difficulty). Each block's hash becomes the next block's prevHash.

Step 3: labels are hard to make (Proof of Work)

A block is accepted only if its hash, read as a 256-bit number, is below a **target** 2^{256-d}, in other words it starts with **d zero bits**. Nobody can predict which header gives such a hash, so the miner changes a counter (the **nonce**) and hashes again, over and over. Each try succeeds with probability $1/2^d$, so a block takes about **2^d guesses** on average, while checking it takes **one** hash.

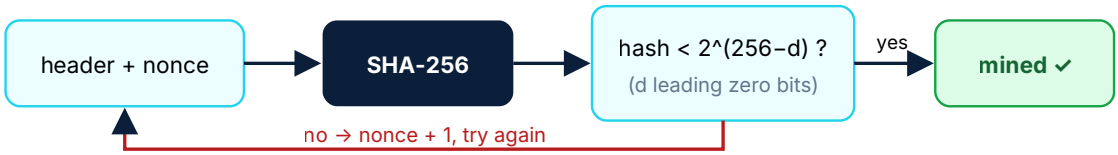


Figure 2 — The mining loop. The only strategy is guessing.

Step 4: why the past becomes (practically) fixed

Edit block 1 and its hash changes, so it no longer meets the target, and block 2's `prevHash` no longer matches. To hide the edit an attacker must re-mine block 1 *and every block after it*, while honest miners keep extending the real chain. The more blocks on top (**confirmations**), the harder it gets. Satoshi's whitepaper (§11) shows the attacker's chance falls exponentially: with 10% of the hash power and 6 blocks behind it is about 0.02%.

Key code walkthrough

1. Unambiguous header encoding (`src/block.js`)

```
// Every field has a fixed type and its own slot: two different headers never give the same bytes.
function serializeHeader(b, dataHash = hashData(b.data)) {
  return JSON.stringify([HEADER_VERSION, b.index, b.prevHash, b.timestamp,
                         b.difficulty, b.nonce, dataHash]);
}
// v1 (tutorial style) — BROKEN: index 1 + timestamp 12 reads exactly like index 11 + timestamp 2
sha256(b.index + b.prevHash + b.timestamp + JSON.stringify(b.data) + b.nonce + b.difficulty)
```

2. Mining with a fast zero-bit check

```
const json = encodeData(data);             // read the caller's payload ONCE (audit E2)
const dataHash = sha256Hex(json);          // payload is fixed while mining: hash it once
header.data = deepFreeze(JSON.parse(json)); // stored data = exactly the bytes that were hashed

for (let nonce = 0; nonce < maxAttempts; nonce++) {  // always terminates (F8)
  header.nonce = nonce;
  const digest = sha256(serializeHeader(header, dataHash));
  if (hasLeadingZeroBits(digest, difficulty)) {
    const block = Object.freeze({ ...header, hash: digest.toString("hex") });
    return { block, attempts: nonce + 1 };
  }
}
```

3. The validator is the security (`src/chain.js`)

```
function checkBlock(block, parent, { difficulty, now }) {
  if (block.hash !== hashBlock(block))           return "block was edited";
  // F2: the RULES decide the difficulty, never the block itself
  if (block.difficulty !== difficulty)           return "wrong difficulty";
  // F1: anyone can recompute a hash; only the target proves work
  if (!meetsDifficulty(block.hash, difficulty))  return "not mined";
  if (parent === null) return null;            // genesis is compared separately (F4)
  if (block.index !== parent.index + 1)          return "wrong index";     // F6
  if (block.prevHash !== parent.hash)            return "chain broken";
  if (block.timestamp < parent.timestamp)        return "earlier than parent";
  if (block.timestamp > now + MAX_FUTURE_MS)     return "over 2 h in the future";
  return null;
}
```

4. Evening improvement: difficulty adjustment

```
// First block of each new window: compare how long the last window took with the target.
const actual = Math.max(1, blocks[height - 1].timestamp - blocks[first].timestamp);
const wanted = gaps * retarget.targetMs;
// ±1 bit = ×2 work; at most ±2 bits (×4) per step, like Bitcoin
const step = clamp(Math.round(Math.log2(wanted / actual)), -2, +2);
return clamp(parentDifficulty + step, 1, 32);
```

Results from the run

A 5-block chain at 16 bits (`npm start`): every hash starts with `0000`, and every prevHash equals the hash above it.

```
#  data                      nonce   hash (first 16 hex)  prevHash
0  Day 02 genesis — Learning 145549  0000d39b694b706e     0000000000000000
1  Sita pays Ram 5 coins      36720  0000ae20c4515493     0000d39b694b706e
2  Ram pays Hari 2 coins      21787  0000059caabc359e     0000ae20c4515493
3  Hari pays Gita 1 coin      96206  0000ae62be3fc208     0000059caabc359e
4  Gita pays Sita 3 coins      5882  0000acbcb8c34e77     0000ae62be3fc208
Mined 4 blocks: 160,599 hashes in 468 ms · Chain valid: true
```

Tampering (`npm run tamper`), changing "5 coins" to "500 coins" in block 1:

```
1. Honest chain                  → VALID
2. Edit block 1                  → INVALID at block 1: hash does not match contents
3. Recompute its hash (no mining) → INVALID at block 1: does not meet the PoW target
4. Re-mine block 1 only          → INVALID at block 2: prevHash does not match
5. Re-mine blocks 1–4            → VALID
Rewriting history cost 491,216 hashes (4 blocks re-mined).
```

Difficulty vs. work (8 fixed samples per level):

Bits	Avg attempts	Expected 2^d	Avg time
4	17	16	0.24 ms
8	262	256	1 ms
12	3,580	4,096	12 ms
16	46,606	65,536	143 ms
20	905,326	1,048,576	2,910 ms

Difficulty adjustment (target 100 ms/block, retarget every 5 blocks):

Blocks	Bits	Avg time
1–5	8	2.1 ms
11–15	12	18.8 ms
21–25	16	163.9 ms
26–30	15	66.6 ms
36–40	16	114.7 ms

Each +4 bits ≈ 16× more work. Attempt counts are identical on every machine; times depend on the CPU (~330,000 hashes/s here). Quality: **40 / 40 tests pass**, ESLint clean, `npm audit` 0 vulnerabilities.

QA & security findings

The first draft followed the common tutorial pattern. An attack script against it, then an independent evening audit, found 14 issues. Each got a failing test first, then a fix.

ID	Severity	What was wrong	Fix
F1	**High**	Validator never checked the target: edited blocks with recomputed, unmined hashes passed.	Require `meetsDifficulty` for every block.
F2	**High**	Difficulty read from the block itself: a forger wrote `difficulty: 1`.	Difficulty comes from the chain's rules.
F3	**Medium**	String concatenation: two different headers, one hash.	Typed JSON-array encoding.
F4	**Medium**	Genesis never checked: whole fake histories accepted.	Block 0 must equal the rule-defined genesis.
F6	**Medium**	Index 99 after 1, timestamps far in the past or future accepted.	index = parent + 1; parent ≤ time ≤ now + 2 h.
F8	**Medium**	Difficulty 256 → infinite mining loop (DoS).	1–32 bits; `maxAttempts`.
E1	**Medium**	`JSON.stringify` merged payloads: `{a: undefined} = {}`, `NaN = null`; a `toJSON()` showing 500 coins hashed as 5.	Strict canonical encoder (sorted keys, plain JSON only).
E2	**Medium**	Chain stored the caller's object: editing your variable rewrote the ledger.	Deep-frozen copy of the exact hashed bytes.
E3–E6	**Low**	Unhashed extra fields accepted; `addBlock` appended invalid blocks; no payload size limit; a bad clock value switched off the future rule.	Exact field set; validate before append; 1 MB limit; strict `now`.
F5, F7	**Low / Info**	Malformed headers hashed as "undefined"; raw BigInt error.	Shape checks; clear errors.

Known issue K1 (low): with difficulty adjustment, miners can fake slow timestamps (time-warp) to lower difficulty, bounded here to −2 bits per window and the 2-hour rule. Bitcoin mitigates this with median-time-past.

Common mistakes

- **Checking "hash = recomputed hash" only.** Anyone can recompute a hash. The *target* is what proves work.
- **Trusting what a block says about itself** (its difficulty, its genesis). Rules come from the protocol, never from the data being checked.
- **Gluing fields together as text** or hashing `JSON.stringify` output. Hash canonical, unambiguous bytes.
- **Validating only from block 1**, or only links and not indexes and timestamps.
- **Thinking Proof of Work makes edits impossible.** It makes them *expensive*; security comes from honest miners holding most of the hash power.

Interview questions

1. Why does editing one old block invalidate every block after it?

The edited block's hash changes (avalanche effect). The next block stores the old hash in `prevHash`, so the link breaks; fixing it changes that block's hash too, and so on to the tip. Each repaired block must also be re-mined.

2. What is "difficulty", and how much work does a block take?

The hash must be below a target 2^{256-d} (d leading zero bits). Each guess succeeds with probability 2^{-d}, so the expected work is 2^d hashes. Today: 16 bits ≈ 65,536 hashes; measured average 46,606 over 8 blocks.

3. Why is Proof of Work "asymmetric"?

Finding a valid nonce needs about 2^d hashes; verifying it needs one. So every node can cheaply check work that was expensive to produce. This idea goes back to Dwork & Naor (1992) and Hashcash (2002).

4. Why must the validator compute the required difficulty itself?

If it trusted the block's own difficulty field, an attacker could declare difficulty 1 and re-mine history almost for free (finding F2). Bitcoin re-derives the target from the previous 2016 blocks' timestamps.

5. Why do exchanges wait for about 6 confirmations?

From Satoshi's §11 formula: an attacker with 10% of the hash power catches up from z = 6 blocks behind with probability ≈ 0.024%, and from z = 1 with ≈ 20%. More confirmations make reversal exponentially less likely.

Exercises summary

Level	Exercise	Key idea
Easy	Where does the chain break? `findBreak(blocks)` returns the first bad block and how many must be re-mined.	Everything after an edit depends on it.
Medium	Implement Satoshi's attacker-success formula; reproduce the whitepaper's tables (q = 0.1, z = 5 → 0.0009137).	Why 6 confirmations.
Challenge	Decode Bitcoin's compact `bits` target (`0x1d00ffff`) and verify the genesis block's PoW with double SHA-256.	Real Bitcoin rules: hash ≤ target, little-endian.

Cheat sheet

- **Block hash** = SHA-256(header); header = version, index, prevHash, time, difficulty, nonce, dataHash
- **Valid PoW:** hash < 2^{256-d} ⇔ d leading zero bits
- **Expected work:** 2^d hashes; +1 bit = ×2, +4 bits = ×16
- **Verify cost:** 1 hash
- **Validator checks:** our genesis · hash matches · difficulty from rules · target met · index +1 · prevHash link · time ≥ parent, ≤ now + 2 h
- **Bitcoin:** double SHA-256, compact `bits` target, retarget every 2016 blocks (×4 max), 10-min blocks
- **Run:** `npm start · npm run tamper · npm test`

References

1. **Bitcoin: A Peer-to-Peer Electronic Cash System**, S. Nakamoto (2008), §3, §4, §11 — https://bitcoin.org/bitcoin.pdf
2. **Bitcoin Developer Guide: Block Chain** — https://developer.bitcoin.org/devguide/block_chain.html
3. **Bitcoin Developer Reference: Block Headers & Target nBits** — https://developer.bitcoin.org/reference/block_chain.html
4. **Mastering Bitcoin, 3rd ed.**, A. Antonopoulos & D. Harding, ch. 11–12 — https://github.com/bitcoinbook/bitcoinbook
5. **Pricing via Processing or Combatting Junk Mail**, C. Dwork & M. Naor (CRYPTO 1992) — https://www.wisdom.weizmann.ac.il/~naor/PAPERS/pvp.pdf
6. **Hashcash: A Denial of Service Counter-Measure**, A. Back (2002) — http://www.hashcash.org/papers/hashcash.pdf
7. **ethereum.org: Proof-of-Work** — https://ethereum.org/en/developers/docs/consensus-mechanisms/pow/
8. **RFC 8785: JSON Canonicalization Scheme** — https://www.rfc-editor.org/rfc/rfc8785
9. **NIST FIPS 180-4: Secure Hash Standard** — https://csrc.nist.gov/pubs/fips/180-4/upd1/final

Code, tests and the full QA report: https://github.com/Khadka1996/Learning-Blockchain/tree/main/day-02